

DarkBox Contracts Audit Report — Ocean Pass

Date: 2026-06-13 UTC

Scope: `packages/contracts`

Contracts reviewed: `DarkBoxBridge`, `ShadowBridgeController`, deploy scripts, tests

Method: local sandboxed Solidity audit workflow + 12 specialist/gap-hunter agent passes + manual context triage.

Executive summary

The first audit pass produced one important lesson: generic escrow invariants do not directly apply to DarkBox.

DarkBox is a prediction-market game. A user may correctly withdraw **more than they deposited** if their agent wins. Therefore a public-bridge rule like `totalDeposited[owner] - totalWithdrawn[owner] >= amount` is wrong for this product. It would break winners.

The real security boundary is:

1. hidden ledger / private market state determines each owner's final withdrawable value;
2. a TEE/CVM-protected signer authorizes only withdrawals backed by that hidden ledger;
3. the public bridge verifies the signer, destination chain/bridge, expiry, and nonce;
4. the public bridge must hold enough global escrow liquidity to pay the signed amount.

Verification status

Command run:

```
cd /home/xiko/darkbox
pnpm --filter @darkbox/contracts test
```

Result after fixes:

```
38 tests passed, 0 failed, 0 skipped
```

Added/confirmed regression coverage:

- constructor rejects zero signer;
- `setSigner(address(0))` rejects;`
- deposits reject short token receipts;
- winner withdrawal can exceed original deposit when signed and vault funded.

Findings

F-01 — Zero signer can brick normal withdrawals

Severity: Medium

Status: Fixed

Contract: `DarkBoxBridge`

Functions: `constructor`, `setSigner`

Issue:

The bridge originally rejected zero admin and zero USDC, but did not reject zero signer. Since every normal withdrawal requires recovering the configured signer, a zero signer would make normal withdrawals impossible until admin intervention.

Fix:

- `constructor` now requires `\_signer != address(0)`.
- `setSigner` now requires `newSigner != address(0)`.
- Tests added.

F-02 — Deposit accounting trusted requested amount instead of actual receipt

Severity: Medium for generic ERC20 / Low for known Base USDC

Status: Fixed

Contract: `DarkBoxBridge`

Function: `deposit`

Issue:

The bridge credited `totalDeposited[beneficiary] += amount` after a successful `transferFrom`. For fee-on-transfer, rebasing, malicious, or otherwise nonstandard tokens, the bridge could receive less than `amount` while still crediting the full nominal amount.

Product context:

Production should use canonical USDC, so this is mainly a deploy/config hardening issue. But testnet/mock environments and accidental wrong-token deployment make it worth fixing.

Fix:

- `deposit` now measures `balanceBefore` / `balanceAfter`.
- If received amount is not exactly requested amount, it reverts with `ShortTokenReceipt(requested, received)`.
- Regression test added with a short-receipt token.

FP-01 — Per-owner deposit cap on withdrawals

Severity: Not a bug / false positive after product-context triage

Status: Reverted from patch, covered by positive test

Contract: `DarkBoxBridge`

Function: `withdraw`

Initial generic-audit claim:

`withdraw` should require $\text{totalDeposited}[\text{owner}] - \text{totalWithdrawn}[\text{owner}] \geq \text{amount}$.

Why this is wrong for DarkBox:

A winning player can withdraw more than they deposited. Losers fund winners through the market. The public bridge does not know the hidden game PnL; it only knows whether a trusted signer authorized the withdrawal.

Correct invariant:

- The TEE/CVM signer must only sign withdrawals that match the hidden ledger after burns/trades/resolution.
- The public bridge must verify signer, destination chain, destination bridge, nonce, deadline, and global token transfer success.
- `totalDeposited` and `totalWithdrawn` are lifetime accounting counters, not per-owner withdrawable caps.

Regression added:

`test\_WithdrawCanExceedOriginalDepositWhenSignedAndVaultFunded` proves a signed winner withdrawal above original deposit succeeds when the vault is funded.

Remaining security work for Dan

TEE signer is the real bridge security boundary

The withdrawal signer must move into TEE/CVM security land. If the signer is compromised, the public bridge can be drained by valid signatures. This is by design for the MVP privacy model, but must be handled seriously.

Required Dan-agent work:

- isolate signer key in Phala/CVM or equivalent;
- authenticate withdrawal requests from the Mini App/backend;
- require hidden-ledger burn/final-state proof before signing;
- log signed withdrawal commands for reveal/replay;
- rate-limit and monitor signer output;
- rotate signer/admin away from hot EOAs before real funds.

Shadow controller/coordinator is privileged

`ShadowBridgeController` trusts its coordinator to map accounts, mint, burn, and lock balances correctly. This is acceptable for MVP private execution, but coordinator compromise is total shadow-ledger compromise.

Required Dan-agent work:

- run coordinator inside the hidden/CVM boundary;
- tie `depositOpId` to public deposit events;
- tie `withdrawalId`, `userCommandHash`, and public bridge nonce into one canonical withdrawal record;
- include all mapping/mint/burn/lock events in reveal artifacts.

Frontier/orderbook audit is blocked

The current repo snapshot does not contain the actual Frontier/orderbook contracts/integration. The audit request includes “run it again on the orderbook too,” but there is nothing concrete in this repo snapshot to audit yet.

Required Dan-agent work:

- pin exact Frontier/orderbook repo/commit;
- vendor or reference the source;
- run the same sandboxed audit workflow over that source;
- include a separate PDF report.

Deployment notes

Base Sepolia deployment completed in this pass:

- `DarkBoxBridge`: `0xe0004c955721b3A994E94CCcA86d91Da4Cf2E6f9`
- `MockERC20` test USDC: `0x8C885Cb844362Ed8d161792aEA6745d29d839246`
- deployer/admin/signer: `0x7c2Af79eD218f75664ae23820C35102Fd8560E6D`

This deployment was rerun after the final audit-context correction and deposit hardening.

AI audit disclaimer

This report was produced by an AI-assisted local audit workflow and manual triage. It is not a substitute for an independent human security review before handling meaningful funds. Treat findings and false positives as engineering evidence to verify, not final authority.